SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

TECHNICAL REPORT

RDD Streaming, Cardinality Sketches, and PageRank Algorithms

AUTHOR

Reza Sanaee Nezhad

Student ID: 401101502

INSTRUCTOR / INSTRUCTORS

Dr Iman Gholampour

ACADEMIC CONTEXT

Big Data Processing

Contents

1 Theory 1: graph model and transition convention 2

2 Theory 2: complete DGIM bucket trace 2

3 Theory 3: FM maximum register 4

4 Theory 4: Bloom-filter derivation 4

5 Theory 5: cooperating spam farms 4

6 Practical 1: file-based stream simulation 4

7 Volume windows and top-three cameras 8

8 Repeated plates and behavioral interpretation 15

9 Camera diversity 21

10 Practical 2: four-hash distinct counting 27

11 Practical 3: directed camera-transition graph 33

12 PageRank with dangling nodes 35

13 Rankings, count comparison, and visualization 38

14 Four-node ranking results 40

1 Theory 1: graph model and transition convention

The graph has A -> B,C,D; B -> A,D; C -> A; D -> B,C. With destination rows and source columns in order A,B,C,D, the column-stochastic matrix is:

Destination/ source	A	B	C	D
A	0	1/2	1	0
B	1/3	0	0	1/2
C	1/3	0	0	1/2
D	1/3	1/2	0	0

Solve r=0.85 M r+0.15 e with the appropriate normalized teleport distribution. B/C/D have equal stationary scores in some of these teleport configurations despite having different outgoing-link structures. The stationary numerical vectors are provided in the numerical calculations section below. Focused teleport favors its seed directly and influences other nodes through the actual graph transitions. TrustRank uses e=(0,1,0,0). Raw spam mass is 1-TR/PR and can be negative for trusted pages. Clipping negative values to zero is an optional display convention and must be stated. HITS authority and hub vectors are eigenvectors associated with incoming/outgoing good-link structure and depend on normalization. PageRank describes a different random-walk model; their orderings need not agree.

$$r_{t+1}(v) = \frac{1-\beta}{N} + \frac{\beta d_t}{N} + \beta \sum_u r_t(u) P(u \to v).$$

2 Theory 2: complete DGIM bucket trace

Process the sixteen-bit sequence in timestamp order, storing at most two buckets per power-of-two size and merging the two oldest equal-size buckets. A bucket timestamp is its newest represented one, not the start of its interval. At t=12 the buckets newest-first are (1,12),(2,11),(2,8),(4,5). At t=13 add (1,13), leaving two unit buckets. t=14 is zero. At t=15 add (1,15); merge the older unit buckets into (2,13), then merge the older size-two buckets into (4,11). At t=16 add (1,16).

Final bucket size	Timestamp	Represented one timestamps
1	16	16
1	15	15
2	13	12,13
4	11	7,8,10,11
4	5	1,3,4,5

For N=12, timestamp 5 is still allowed at t=16, although that oldest bucket includes earlier ones. For K=7 the query covers t=10..16. Sum the fully included newest buckets and half the boundary bucket: 1+1+2+4/2=6. The exact last-seven count is also six, giving absolute error zero and relative error zero. Exact agreement in this example does not make DGIM an exact counter in general.

Table 3: Theory 2: complete DGIM bucket trace - numerical data and parameters.

Bucket	**Size**	**Timestamp**	**Represented one timestamps**
1	1	16	16
2	1	15	15
3	2	13	12, 13
4	4	11	7, 8, 10, 11
5	4	5	1, 3, 4, 5

3 Theory 3: FM maximum register

The six hash strings have trailing-zero counts 3,5,4,3,3,6; R=6 and the simple estimate is 64. A nine-bit all-zero string gives a capped trailing-zero count of 9 and estimate 512 under the exercise convention. A finite-width implementation must explicitly handle zero hashes instead of looping indefinitely. One unusually long zero suffix can double the estimate for every additional register bit. Therefore the estimate can overshoot or undershoot the true count, and adding one element need not reduce error. Uniform hashes give a suffix probability of about 2^(-k), explaining the noisy maximum statistic.

4 Theory 4: Bloom-filter derivation

Each partition has n/k bits and receives one hash per inserted element. A selected bit remains zero with probability (1-k/n)^m, approximately exp(-mk/n). Under independent partitions, false-positive probability is [1-(1-k/n)^m]^k, approximately (1-exp(-mk/n))^k. A conventional single-array filter has a closely related approximation, not an exactly identical finite-size occupancy model. Minimize the log of the approximation in continuous k to obtain k=(n/m) ln 2; choose a nearby positive integer and require valid partition sizes. Independence and uniformity are modeling assumptions.

5 Theory 5: cooperating spam farms

The submission proposes mutual target-page links while support pages point to their own target. A mutual link can retain or redirect some probability within the combined farm. However each new outgoing link also divides the source target's outgoing probability, and benefits depend on support links, external inflow, and the recorded farm model. Without those constraints, the proposed connection is a candidate strategy, not a proved universal PageRank-maximizing topology.

6 Practical 1: file-based stream simulation

A Python feeder reads traffic.csv in chunks of 5000 records and publishes a new CSV file every five seconds. Spark's textFileStream watches the input directory and detects newly created files. Files should be written completely and atomically renamed into the watched directory so that a partial file is not consumed. Replaying a static dataset creates a simulated arrival stream, not live camera telemetry. The StreamingContext uses a five-second batch interval. Parsing extracts camera from index 0, passage time from index 4, and plate from index 6. Console samples demonstrate newly arriving tuples in successive batches. Splitting on commas assumes simple unquoted fields; a CSV-aware parser is needed when fields may contain quoted commas. The screenshots document this classroom run without establishing exactly-once delivery or restart recovery.

```python
import time
import os

SOURCE_FILE = "/home/reza/HW4/traffic.csv"
DEST_DIR = "/home/reza/HW4/stream_input"
CHUNK_SIZE = 5000
INTERVAL = 5

def simulate_streaming():
    if not os.path.exists(DEST_DIR):
        os.makedirs(DEST_DIR)

    with open(SOURCE_FILE, 'r') as f:
        header = f.readline()
        batch_lines = []
        batch_counter = 1

        for line in f:
            batch_lines.append(line)

            if len(batch_lines) >= CHUNK_SIZE:
                output_file = f"{DEST_DIR}/batch_{batch_counter}.csv"
                with open(output_file, 'w') as out_f:
                    out_f.writelines(batch_lines)

                print(f"Batch {batch_counter} moved.")
                batch_lines = []
                batch_counter += 1
                time.sleep(INTERVAL)

        if batch_lines:
            output_file = f"{DEST_DIR}/batch_{batch_counter}.csv"
            with open(output_file, 'w') as out_f:
                out_f.writelines(batch_lines)
            print(f"Final Batch {batch_counter} moved.")

if __name__ == "__main__":
    simulate_streaming()
```

Figure 1: Implementation and output: Practical 1: file-based stream simulation.

```
Batch 25 moved.
Batch 26 moved.
Batch 27 moved.
Batch 28 moved.
Batch 29 moved.
Batch 30 moved.
Batch 31 moved.
Batch 32 moved.
Batch 33 moved.
Batch 34 moved.
Batch 35 moved.
Batch 36 moved.
Batch 37 moved.
Batch 38 moved.
Batch 39 moved.
Batch 40 moved.
Batch 41 moved.
Batch 42 moved.
```

Figure 2: Diagram or recorded result: Practical 1: file-based stream simulation.

```python
import sys
import os
from pyspark import SparkContext, SparkConf
from pyspark.streaming import StreamingContext

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

conf = SparkConf().setAppName("TrafficStreamingAnalysis").setMaster("local[2]")

try:
    sc.stop()
except:
    pass

sc = SparkContext(conf=conf)
sc.setLogLevel("WARN")

ssc = StreamingContext(sc, 5)

input_directory = "file:///home/reza/HW4/stream_input"
dstream = ssc.textFileStream(input_directory)

def parse_traffic_line(line):
    try:
        parts = line.split(',')
        device_code = parts[0].strip()
        pass_time = parts[4].strip()
        plate_index = parts[6].strip()
        return (device_code, pass_time, plate_index)
    except:
        return None

parsed_dstream = dstream.map(parse_traffic_line).filter(lambda x: x is not None)

parsed_dstream.pprint()

ssc.start()
```

Figure 3: Implementation and output: Practical 1: file-based stream simulation.

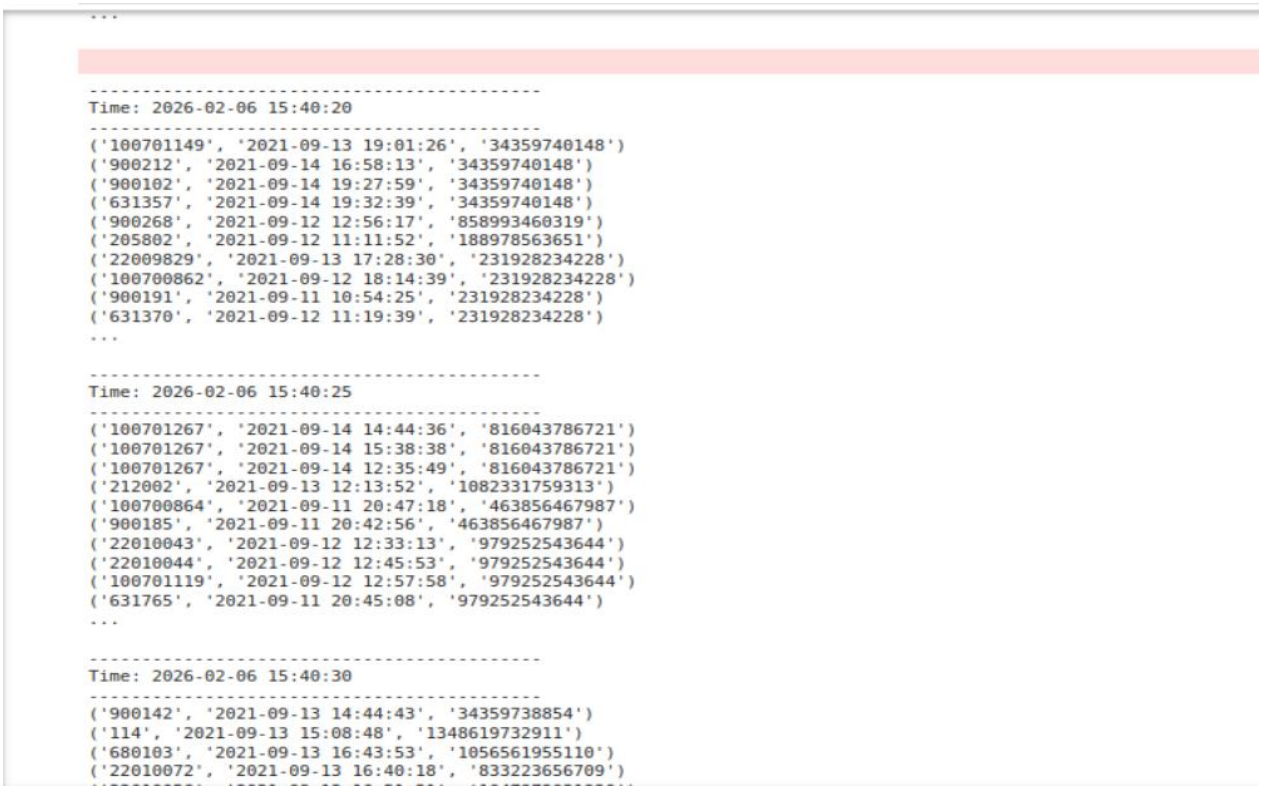

Figure 4: Diagram or recorded result: Practical 1: file-based stream simulation.

7 Volume windows and top-three cameras

A thirty-second window advances every ten seconds, covering six five-second batches when full. At the configured feeder rate, a full window can contain about 30000 records. Early counts rise while the window fills; later counts reflect replay-arrival time, not necessarily a real thirty-second interval in the recorded timestamps. reduceByKey obtains counts by camera, and sorting selects the top three. The submitted examples repeatedly identify cameras 900212 and 100700853 among high-count cameras, with sample counts rising from about 200 toward 400 as the window fills. These counts should not be presented as a measured physical camera flow rate because the feeder compresses/replays the data. A larger window smooths fluctuations but retains older arrivals; a smaller window reacts faster but is noisier. A thirty-second window does not guarantee a thirty-second maximum detection delay because triggers, backlog, and processing also contribute. Ranking agreement with an earlier static analysis is a consistency observation, not an independent correctness proof.

```
per_camera_counts = windowed_dstream.map(lambda x: (x[0], 1)).reduceByKey(lambda a, b: a + b)

sorted_camera_counts = per_camera_counts.transform(lambda rdd: rdd.sortBy(lambda x: x[1], ascending=False))

sorted_camera_counts.pprint(10)

ssc.start()
```

Figure 5: Implementation and output: Volume windows and top-three cameras.

```python
import sys
import os
from pyspark import SparkContext, SparkConf
from pyspark.streaming import StreamingContext

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

conf = SparkConf().setAppName("TrafficWindowAnalysis").setMaster("local[2]")

try:
    sc.stop()
except:
    pass

sc = SparkContext(conf=conf)
sc.setLogLevel("WARN")

ssc = StreamingContext(sc, 5)

input_directory = "file:///home/reza/HW4/stream_input"
dstream = ssc.textFileStream(input_directory)

def parse_traffic_line(line):
    try:
        parts = line.split(',')
        device_code = parts[0].strip()
        pass_time = parts[4].strip()
        plate_index = parts[6].strip()
        return (device_code, pass_time, plate_index)
    except:
        return None

parsed_dstream = dstream.map(parse_traffic_line).filter(lambda x: x is not None)

WINDOW_DURATION = 30
SLIDE_DURATION = 10

windowed_dstream = parsed_dstream.window(WINDOW_DURATION, SLIDE_DURATION)

def print_total_count(rdd):
    count = rdd.count()
    if count > 0:
        print(f"--- Total Vehicles in last {WINDOW_DURATION} seconds: {count} ---")

windowed_dstream.foreachRDD(print_total_count)
```

Figure 6: Implementation and output: Volume windows and top-three cameras.

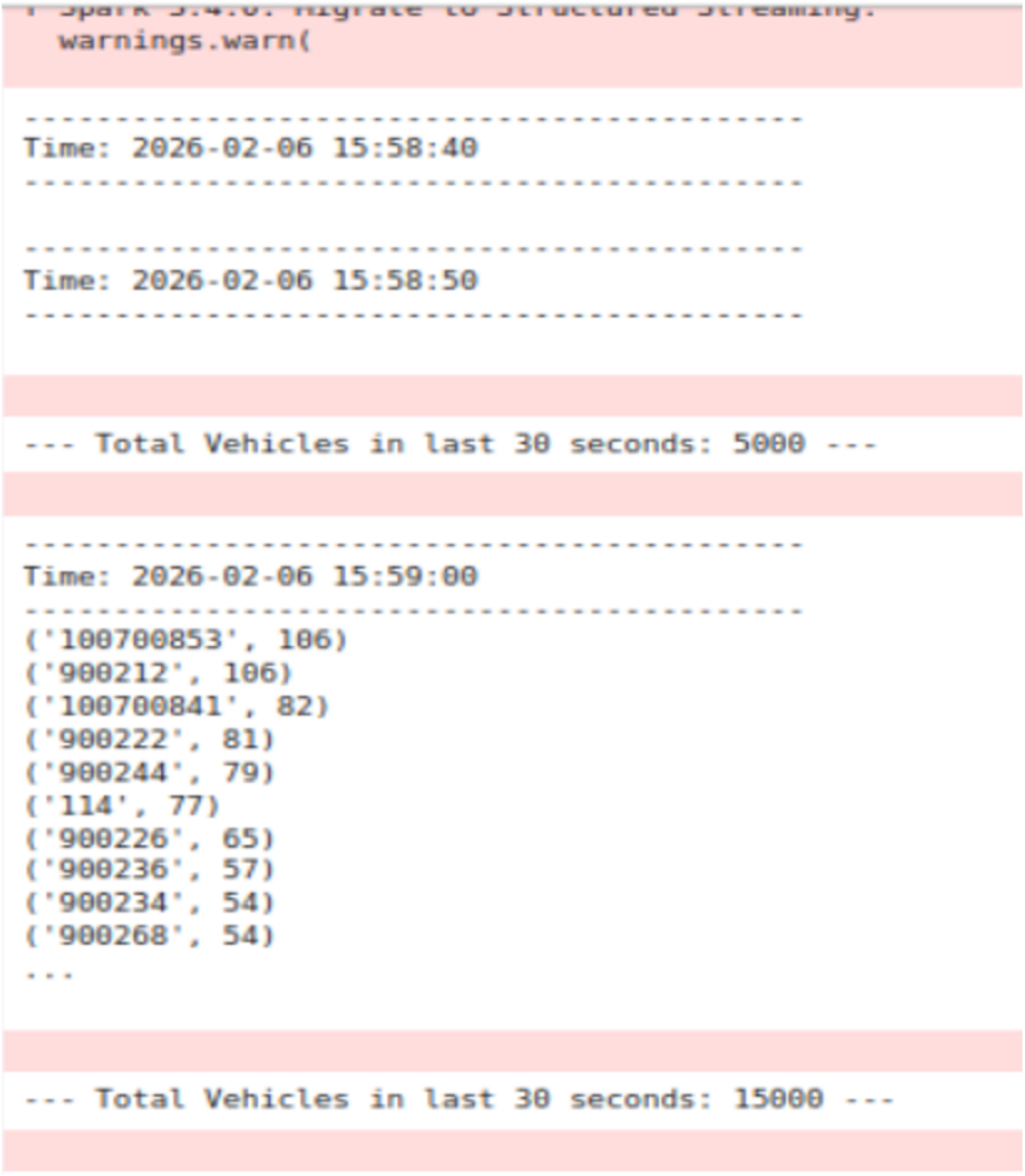

Figure 7: Diagram or recorded result: Volume windows and top-three cameras. Panel 1 of 2.

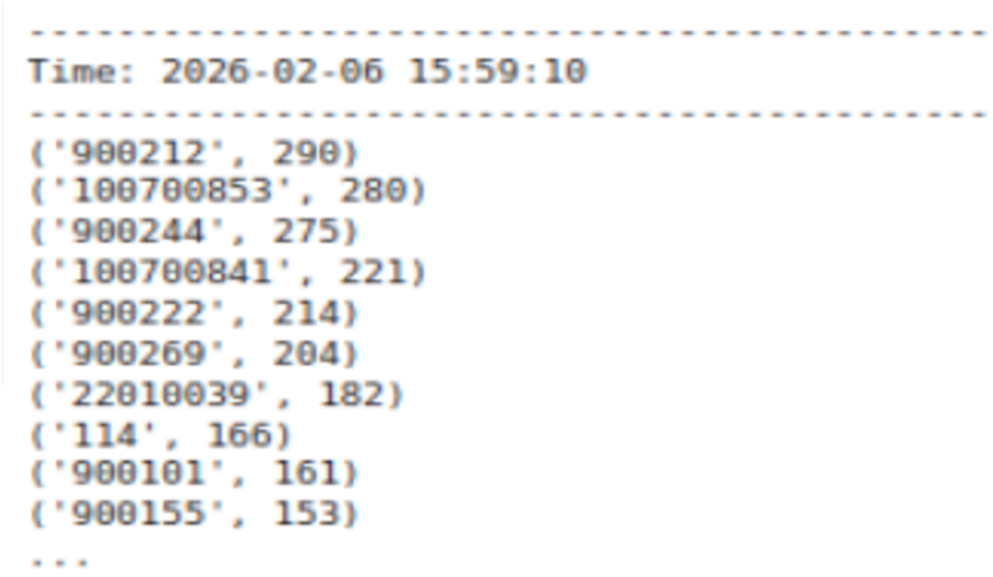

Figure 8: Diagram or recorded result: Volume windows and top-three cameras. Panel 2 of 2.

```
--- Total Vehicles in last 30 seconds: 25000 ---

------------------------------------------
Time: 2026-02-06 15:59:20
------------------------------------------
('900212', 554)
('900244', 482)
('100700853', 477)
('900222', 388)
('100700841', 384)
('900269', 379)
('900142', 310)
('900101', 282)
('900268', 272)
('22010039', 266)
...

--- Total Vehicles in last 30 seconds: 30000 ---

------------------------------------------
Time: 2026-02-06 15:59:30
------------------------------------------
('900212', 682)
('900244', 646)
('100700853', 533)
('900222', 495)
('900269', 486)
('100700841', 481)
('900142', 406)
('900101', 375)
('900155', 340)
('900268', 314)
...
```

Figure 9: Diagram or recorded result: Volume windows and top-three cameras.

```
--- Total Vehicles in last 30 seconds: 30000 ---
-----------------------------------------------
Time: 2026-02-06 15:59:40
-----------------------------------------------
('900212', 626)
('900244', 585)
('100700853', 509)
('100700841', 488)
('900269', 468)
('900222', 459)
('900142', 401)
('900101', 332)
('900225', 322)
('900155', 314)
...
```

```
--- Total Vehicles in last 30 seconds: 30000 ---
-----------------------------------------------
Time: 2026-02-06 15:59:50
-----------------------------------------------
('900244', 579)
('900212', 565)
('100700841', 503)
('100700853', 485)
('900269', 467)
('900222', 425)
('900142', 363)
('900101', 362)
('22010039', 340)
('900225', 338)
...
```

Figure 10: Diagram or recorded result: Volume windows and top-three cameras.

```
def process_rdd(time, rdd):
    try:
        if not rdd.isEmpty():
            top_3 = rdd.take(3)
            print(f"========= Time: {time} =========")
            print(f"Top 3 Busiest Cameras in last {WINDOW_DURATION} seconds:")
            for rank, (camera, count) in enumerate(top_3, 1):
                print(f"{rank}. Camera {camera}: {count} vehicles")
    except:
        pass

per_camera_counts = windowed_dstream.map(lambda x: (x[0], 1)).reduceByKey(lambda a, b: a + b)

sorted_camera_counts = per_camera_counts.transform(lambda rdd: rdd.sortBy(lambda x: x[1], ascending=False))

sorted_camera_counts.foreachRDD(process_rdd)

ssc.start()
```

Figure 11: Implementation and output: Volume windows and top-three cameras.

```
import sys
import os
from pyspark import SparkContext, SparkConf
from pyspark.streaming import StreamingContext

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

conf = SparkConf().setAppName("PeakTrafficAnalysis").setMaster("local[2]")

try:
    sc.stop()
except:
    pass

sc = SparkContext(conf=conf)
sc.setLogLevel("WARN")

ssc = StreamingContext(sc, 5)

input_directory = "file:///home/reza/HW4/stream_input"
dstream = ssc.textFileStream(input_directory)

def parse_traffic_line(line):
    try:
        parts = line.split(',')
        device_code = parts[0].strip()
        pass_time = parts[4].strip()
        plate_index = parts[6].strip()
        return (device_code, pass_time, plate_index)
    except:
        return None

parsed_dstream = dstream.map(parse_traffic_line).filter(lambda x: x is not None)

WINDOW_DURATION = 30
SLIDE_DURATION = 10

windowed_dstream = parsed_dstream.window(WINDOW_DURATION, SLIDE_DURATION)
```

Figure 12: Implementation and output: Volume windows and top-three cameras.

8 Repeated plates and behavioral interpretation

The behavioral query uses a sixty-second window, a ten-second slide, and threshold N=3: plates with at least four recorded observations qualify. N is a parameter. Longer windows capture more repeated observations but require more distinct-key state and can combine unrelated events. Shorter windows can separate occurrences that a longer interval would group. Repeated plate observations do not prove multiple distinct camera visits, speeding, prohibited travel, or suspicious intent. Those interpretations require validated identities, locations, travel times, and camera deduplication. The recorded physical-distance explanation is especially inappropriate when timestamps are replayed at a different wall-clock rate.

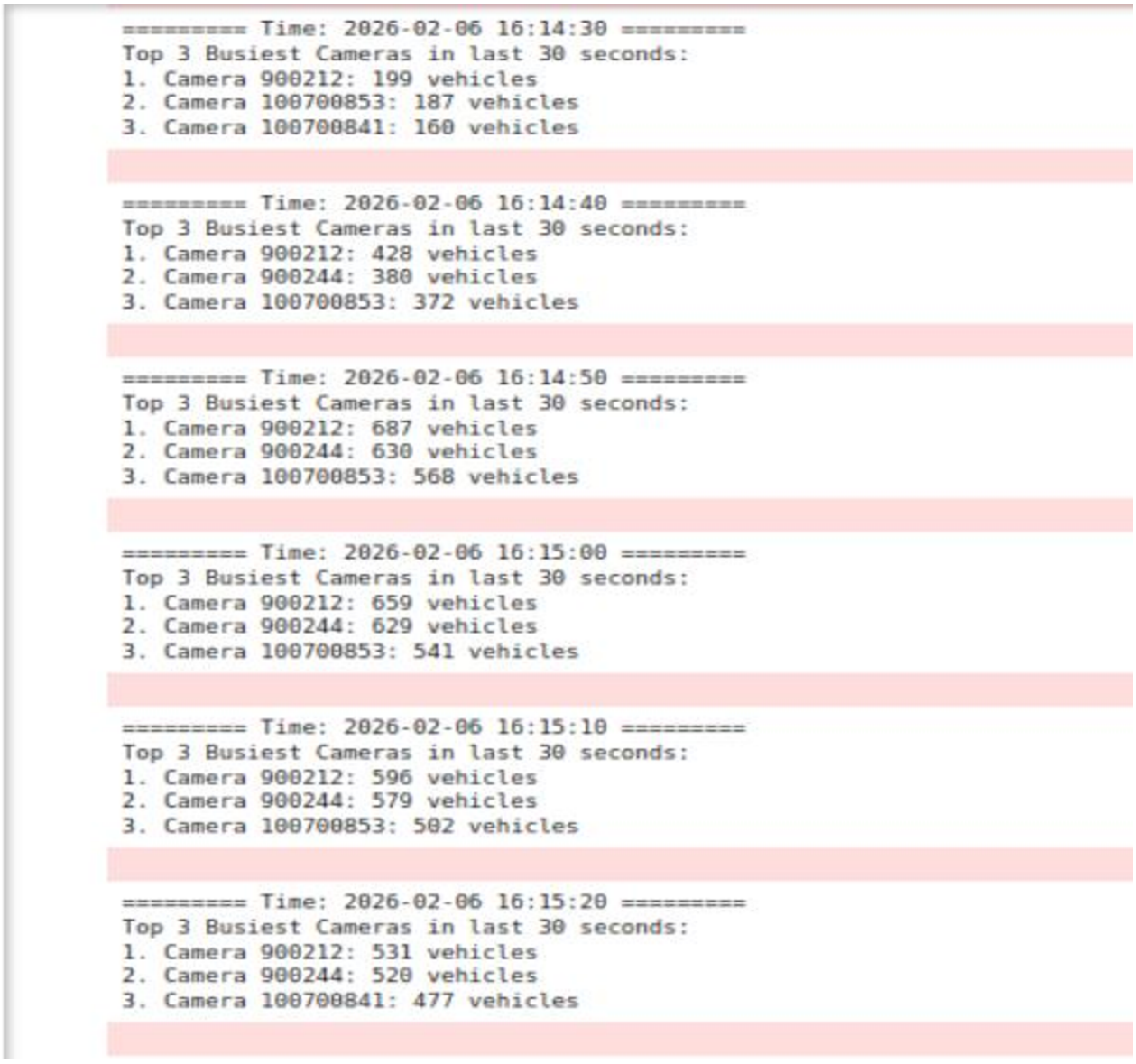

Figure 13: Diagram or recorded result: Repeated plates and behavioral interpretation.

```
def process_frequent_cars(time, rdd):
    try:
        if not rdd.isEmpty():
            frequent_cars = rdd.map(lambda x: (x, 1)) \
                               .reduceByKey(lambda a, b: a + b) \
                               .filter(lambda x: x[1] > THRESHOLD_N) \
                               .sortBy(lambda x: x[1], ascending=False) \
                               .take(20)

            print(f"========= Time: {time} =========")
            print(f"Cars seen more than {THRESHOLD_N} times in last {WINDOW_DURATION} seconds:")
            for rank, (plate, count) in enumerate(frequent_cars, 1):
                print(f"{rank}. Plate {plate}: {count} sightings")
    except:
        pass

windowed_plates.foreachRDD(process_frequent_cars)

ssc.start()
```

Figure 14: Implementation and output: Repeated plates and behavioral interpretation.

```
import sys
import os
from pyspark import SparkContext, SparkConf
from pyspark.streaming import StreamingContext

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

conf = SparkConf().setAppName("FrequentCarAnalysis").setMaster("local[2]")

try:
    sc.stop()
except:
    pass

sc = SparkContext(conf=conf)
sc.setLogLevel("WARN")

ssc = StreamingContext(sc, 5)

input_directory = "file:///home/reza/HW4/stream_input"
dstream = ssc.textFileStream(input_directory)

def parse_plate(line):
    try:
        parts = line.split(',')
        plate_index = parts[6].strip()
        return plate_index
    except:
        return None

plate_dstream = dstream.map(parse_plate).filter(lambda x: x is not None)

WINDOW_DURATION = 60
SLIDE_DURATION = 10
THRESHOLD_N = 3

windowed_plates = plate_dstream.window(WINDOW_DURATION, SLIDE_DURATION)
```

Figure 15: Implementation and output: Repeated plates and behavioral interpretation.

```
========== Time: 2026-02-06 16:44:45 ==========
Cars seen more than 3 times in last 60 seconds:
1. Plate 1675037246756: 160 sightings
2. Plate 377957124396: 135 sightings
3. Plate 14602888881124: 98 sightings
4. Plate 919123003161: 83 sightings
5. Plate 1546188227913: 79 sightings
6. Plate 1185410976213: 76 sightings
7. Plate 1116691497606: 75 sightings
8. Plate 343597385400: 69 sightings
9. Plate 1649267441891: 67 sightings
10. Plate 1022202216477: 64 sightings
11. Plate 1357209666721: 62 sightings
12. Plate 128849019238: 60 sightings
13. Plate 1408749274656: 59 sightings
14. Plate 1022202218947: 57 sightings
15. Plate 317827580541: 56 sightings
16. Plate 1554778161780: 54 sightings
17. Plate 85899346312: 53 sightings
18. Plate 790273983502: 51 sightings
19. Plate 498216207598: 49 sightings
20. Plate 532575945078: 47 sightings
```

Figure 16: Diagram or recorded result: Repeated plates and behavioral interpretation.

```
f Spark 3.4.0. Migrate to Structured Streaming.
  warnings.warn(

========= Time: 2026-02-06 16:44:35 =========
Cars seen more than 3 times in last 60 seconds:
1. Plate 1675037246756: 160 sightings
2. Plate 343597385400: 69 sightings
3. Plate 1022202216477: 64 sightings
4. Plate 1357209666721: 62 sightings
5. Plate 1022202218947: 57 sightings
6. Plate 790273983502: 51 sightings
7. Plate 532575945078: 47 sightings
8. Plate 755914244343: 45 sightings
9. Plate 541165879488: 45 sightings
10. Plate 1382979471151: 42 sightings
11. Plate 317827580077: 41 sightings
12. Plate 240518170036: 41 sightings
13. Plate 506806142908: 41 sightings
14. Plate 1194000908822: 41 sightings
15. Plate 1228360648313: 38 sightings
16. Plate 584115554232: 37 sightings
17. Plate 996432413329: 35 sightings
18. Plate 893353198728: 35 sightings
19. Plate 1520418423174: 35 sightings
20. Plate 25769806403: 35 sightings
```

Figure 17: Diagram or recorded result: Repeated plates and behavioral interpretation.

```
========== Time: 2026-02-06 16:44:55 ==========
Cars seen more than 3 times in last 60 seconds:
1. Plate 16750372467 56: 160 sightings
2. Plate 17093969864 59: 158 sightings
3. Plate 377957124396: 135 sightings
4. Plate 11682311048 24: 107 sightings
5. Plate 14602888811 24: 98 sightings
6. Plate 919123003161: 83 sightings
7. Plate 11338713664 44: 81 sightings
8. Plate 15461882279 13: 79 sightings
9. Plate 481036339789: 76 sightings
10. Plate 11854109762 13: 76 sightings
11. Plate 11166914976 06: 75 sightings
12. Plate 343597385400: 69 sightings
13. Plate 16492674418 91: 67 sightings
14. Plate 10222022164 77: 64 sightings
15. Plate 481036339253: 64 sightings
16. Plate 111669152028: 63 sightings
17. Plate 721554506560: 62 sightings
18. Plate 13572096667 21: 62 sightings
19. Plate 128849019238: 60 sightings
20. Plate 14087492746 56: 59 sightings
```

Figure 18: Diagram or recorded result: Repeated plates and behavioral interpretation.

```
========= Time: 2026-02-06 16:45:05 =========
Cars seen more than 3 times in last 60 seconds:
1. Plate 790273982754: 171 sightings
2. Plate 1675037246756: 160 sightings
3. Plate 1709396986459: 158 sightings
4. Plate 377957124396: 135 sightings
5. Plate 1503238556277: 117 sightings
6. Plate 309237646631: 111 sightings
7. Plate 1168231104824: 107 sightings
8. Plate 1460288881124: 98 sightings
9. Plate 549755815293: 84 sightings
10. Plate 919123003161: 83 sightings
11. Plate 1133871366444: 81 sightings
12. Plate 1546188227913: 79 sightings
13. Plate 1185410976213: 76 sightings
14. Plate 481036339789: 76 sightings
15. Plate 1116691497606: 75 sightings
16. Plate 798863917458: 71 sightings
17. Plate 1082331758937: 70 sightings
18. Plate 343597385400: 69 sightings
19. Plate 257698037971: 68 sightings
20. Plate 1649267441891: 67 sightings
```

Figure 19: Diagram or recorded result: Repeated plates and behavioral interpretation.

9 Camera diversity

A sixty-second window with a twenty-second slide computes total observations per camera and distinct (camera,plate) pairs. Joining these counts gives diversity=UniquePlates/TotalPasses, between zero and one for nonempty windows. A low ratio indicates repeat observations, which may reflect congestion, duplicate detections, loops, parking movements, or sensor behavior. It does not uniquely identify a road type. The report filters TotalPasses>100 and diversity<0.6. Its example camera 100701262 has 155 observations and 33 distinct plates: 33/155=0.2129032, or about 4.697 observations per distinct plate. A low-diversity camera is not proven to be outside a highway. Changing window length changes which repeats are grouped; more data can help interpretation while also reducing temporal specificity.

```
import sys
import os
from pyspark import SparkContext, SparkConf
from pyspark.streaming import StreamingContext

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

conf = SparkConf().setAppName("DiversityAnalysis").setMaster("local[2]")

try:
    sc.stop()
except:
    pass

sc = SparkContext(conf=conf)
sc.setLogLevel("WARN")

ssc = StreamingContext(sc, 5)

input_directory = "file:///home/reza/HW4/stream_input"
dstream = ssc.textFileStream(input_directory)

def parse_device_plate(line):
    try:
        parts = line.split(',')
        device_code = parts[0].strip()
        plate_index = parts[6].strip()
        return (device_code, plate_index)
    except:
        return None


device_plate_dstream = dstream.map(parse_device_plate).filter(lambda x: x is not None)

WINDOW_DURATION = 60
SLIDE_DURATION = 20
windowed_dstream = device_plate_dstream.window(WINDOW_DURATION, SLIDE_DURATION)
```

Figure 20: Implementation and output: Camera diversity.

```python
def process_diversity(time, rdd):
    try:
        if not rdd.isEmpty():
            total_counts = rdd.map(lambda x: (x[0], 1)).reduceByKey(lambda a, b: a + b)
            unique_counts = rdd.distinct().map(lambda x: (x[0], 1)).reduceByKey(lambda a, b: a + b)
            stats = total_counts.join(unique_counts)
            results = stats.mapValues(lambda x: (x[0], x[1], x[1]/x[0])) \
                           .map(lambda x: (x[0], x[1][0], x[1][1], x[1][2])) \
                           .sortBy(lambda x: x[1], ascending=False) \
                           .take(15)

            print(f"========= Time: {time} =========")
            print(f"{'Camera':<15} | {'Total Passes':<15} | {'Unique Cars':<15} | {'Diversity Ratio':<15}")
            print("-" * 70)
            for cam, total, unique, ratio in results:
                print(f"{cam:<15} | {total:<15} | {unique:<15} | {ratio:.4f}")
    except:
        pass

windowed_dstream.foreachRDD(process_diversity)

ssc.start()
```

Figure 21: Implementation and output: Camera diversity.

```
  warnings.warn(

========= Time: 2026-02-06 17:04:40 =========
Camera          | Total Passes    | Unique Cars     | Diversity Ratio
----------------------------------------------------------------------
900212          | 290             | 186             | 0.6414
100700853       | 280             | 153             | 0.5464
900244          | 275             | 171             | 0.6218
100700841       | 221             | 157             | 0.7104
900222          | 214             | 145             | 0.6776
900269          | 204             | 142             | 0.6961
22010039        | 182             | 131             | 0.7198
114             | 166             | 83              | 0.5000
900101          | 161             | 97              | 0.6025
900155          | 153             | 105             | 0.6863
900142          | 151             | 104             | 0.6887
900236          | 150             | 87              | 0.5800
22010040        | 148             | 101             | 0.6824
900268          | 140             | 86              | 0.6143
900225          | 139             | 95              | 0.6835

========= Time: 2026-02-06 17:05:00 =========
Camera          | Total Passes    | Unique Cars     | Diversity Ratio
----------------------------------------------------------------------
900212          | 788             | 488             | 0.6193
900244          | 725             | 456             | 0.6290
100700853       | 639             | 377             | 0.5900
900222          | 576             | 363             | 0.6302
100700841       | 563             | 402             | 0.7140
900269          | 533             | 344             | 0.6454
900142          | 459             | 288             | 0.6275
900101          | 425             | 272             | 0.6400
900155          | 387             | 258             | 0.6667
900268          | 368             | 211             | 0.5734
900225          | 358             | 223             | 0.6229
22010039        | 350             | 260             | 0.7429
900207          | 342             | 208             | 0.6082
900246          | 340             | 222             | 0.6529
900236          | 328             | 201             | 0.6128
```

Figure 22: Diagram or recorded result: Camera diversity.

```
========= Time: 2026-02-06 17:05:20 =========
Camera          | Total Passes    | Unique Cars     | Diversity Ratio
--------------------------------------------------------------------------
900212          | 1119            | 731             | 0.6533
900244          | 1061            | 698             | 0.6579
100700853       | 962             | 609             | 0.6331
100700841       | 887             | 640             | 0.7215
900269          | 846             | 558             | 0.6596
900222          | 813             | 517             | 0.6359
900142          | 673             | 454             | 0.6746
900101          | 644             | 403             | 0.6258
22010039        | 606             | 462             | 0.7624
900225          | 583             | 385             | 0.6604
900268          | 574             | 329             | 0.5732
900155          | 558             | 389             | 0.6971
900246          | 509             | 332             | 0.6523
22010040        | 503             | 357             | 0.7097
900107          | 502             | 369             | 0.7351

========= Time: 2026-02-06 17:05:40 =========
Camera          | Total Passes    | Unique Cars     | Diversity Ratio
--------------------------------------------------------------------------
900212          | 1253            | 803             | 0.6409
900244          | 1212            | 792             | 0.6535
100700853       | 1022            | 681             | 0.6663
900269          | 993             | 624             | 0.6284
900222          | 958             | 581             | 0.6065
100700841       | 952             | 665             | 0.6985
900101          | 756             | 479             | 0.6336
900142          | 753             | 501             | 0.6653
900225          | 656             | 441             | 0.6723
900155          | 653             | 446             | 0.6830
900268          | 650             | 363             | 0.5585
22010039        | 644             | 489             | 0.7593
900246          | 597             | 389             | 0.6516
900107          | 588             | 403             | 0.6854
22010040        | 582             | 407             | 0.6993
```

Figure 23: Diagram or recorded result: Camera diversity.

```python
import sys
import os
from pyspark import SparkContext, SparkConf
from pyspark.streaming import StreamingContext

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

conf = SparkConf().setAppName("CameraBehaviorComparison").setMaster("local[2]")

try:
    sc.stop()
except:
    pass

sc = SparkContext(conf=conf)
sc.setLogLevel("WARN")

ssc = StreamingContext(sc, 5)

input_directory = "file:///home/reza/HW4/stream_input"
dstream = ssc.textFileStream(input_directory)

def parse_device_plate(line):
    try:
        parts = line.split(',')
        device_code = parts[0].strip()
        plate_index = parts[6].strip()
        return (device_code, plate_index)
    except:
        return None

device_plate_dstream = dstream.map(parse_device_plate).filter(lambda x: x is not None)
```

Figure 24: Implementation and output: Camera diversity.

```python
WINDOW_DURATION = 60
SLIDE_DURATION = 20
windowed_dstream = device_plate_dstream.window(WINDOW_DURATION, SLIDE_DURATION)

MIN_TRAFFIC_COUNT = 100
MAX_DIVERSITY_RATIO = 0.60

def analyze_camera_behavior(time, rdd):
    try:
        if not rdd.isEmpty():

            total_counts = rdd.map(lambda x: (x[0], 1)).reduceByKey(lambda a, b: a + b)
            unique_counts = rdd.distinct().map(lambda x: (x[0], 1)).reduceByKey(lambda a, b: a + b)
            stats = total_counts.join(unique_counts)
            metrics = stats.mapValues(lambda x: (x[0], x[1], x[1]/x[0]))
            suspicious_cameras = metrics.filter(lambda x: x[1][0] > MIN_TRAFFIC_COUNT and x[1][2] < MAX_DIVERSITY_RATIO) \
                                        .sortBy(lambda x: x[1][2], ascending=True) \
                                        .take(10)

            print(f"========= Time: {time} =========")
            print(f"Cameras with HIGH Traffic (> {MIN_TRAFFIC_COUNT}) and LOW Diversity (< {MAX_DIVERSITY_RATIO}):")
            print(f"{'Camera':<15} | {'Total':<10} | {'Unique':<10} | {'Ratio':<10} | {'Status'}")
            print("-" * 75)

            if not suspicious_cameras:
                print("No cameras match the criteria in this window.")
            else:
                for cam, (total, unique, ratio) in suspicious_cameras:
                    print(f"{cam:<15} | {total:<10} | {unique:<10} | {ratio:.4f}     | CONGESTED/LOOP")

            print("\n")
    except:
        pass

windowed_dstream.foreachRDD(analyze_camera_behavior)

ssc.start()
```

Figure 25: Implementation and output: Camera diversity.

```
========== Time: 2026-02-06 17:24:50 ==========
Cameras with HIGH Traffic (> 100) and LOW Diversity (< 0.6):
Camera          | Total      | Unique     | Ratio      | Status
--------------------------------------------------------------------------------
100701266       | 196        | 52         | 0.2653     | CONGESTED/LOOP
100701262       | 248        | 90         | 0.3629     | CONGESTED/LOOP
22010086        | 241        | 93         | 0.3859     | CONGESTED/LOOP
100701252       | 198        | 77         | 0.3889     | CONGESTED/LOOP
22010083        | 190        | 89         | 0.4684     | CONGESTED/LOOP
100701253       | 114        | 54         | 0.4737     | CONGESTED/LOOP
900272          | 178        | 85         | 0.4775     | CONGESTED/LOOP
900199          | 177        | 86         | 0.4859     | CONGESTED/LOOP
22010044        | 157        | 78         | 0.4968     | CONGESTED/LOOP
22010094        | 168        | 84         | 0.5000     | CONGESTED/LOOP
```

```
========== Time: 2026-02-06 17:25:10 ==========
Cameras with HIGH Traffic (> 100) and LOW Diversity (< 0.6):
Camera          | Total      | Unique     | Ratio      | Status
--------------------------------------------------------------------------------
100701266       | 225        | 52         | 0.2311     | CONGESTED/LOOP
22010086        | 429        | 113        | 0.2634     | CONGESTED/LOOP
100701262       | 313        | 103        | 0.3291     | CONGESTED/LOOP
100701252       | 274        | 94         | 0.3431     | CONGESTED/LOOP
22010085        | 297        | 107        | 0.3603     | CONGESTED/LOOP
22010043        | 106        | 39         | 0.3679     | CONGESTED/LOOP
22010083        | 254        | 110        | 0.4331     | CONGESTED/LOOP
100701263       | 135        | 60         | 0.4444     | CONGESTED/LOOP
100701267       | 209        | 97         | 0.4641     | CONGESTED/LOOP
100701253       | 157        | 73         | 0.4650     | CONGESTED/LOOP
```

Figure 26: Diagram or recorded result: Camera diversity.

```
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/02/06 17:23:45 WARN NativeCodeLoader: Unable to load native-hadoop library for your platfor
le
/home/reza/miniconda3/envs/spark_fix/lib/python3.10/site-packages/pyspark/streaming/context.py
f Spark 3.4.0. Migrate to Structured Streaming.
  warnings.warn(

========== Time: 2026-02-06 17:24:10 ==========
Cameras with HIGH Traffic (> 100) and LOW Diversity (< 0.6):
Camera          | Total      | Unique     | Ratio      | Status
--------------------------------------------------------------------------------
114             | 122        | 52         | 0.4262     | CONGESTED/LOOP
900101          | 117        | 63         | 0.5385     | CONGESTED/LOOP
900236          | 111        | 63         | 0.5676     | CONGESTED/LOOP
100700853       | 187        | 111        | 0.5936     | CONGESTED/LOOP
```

```
========== Time: 2026-02-06 17:24:30 ==========
Cameras with HIGH Traffic (> 100) and LOW Diversity (< 0.6):
Camera          | Total      | Unique     | Ratio      | Status
--------------------------------------------------------------------------------
100701262       | 155        | 33         | 0.2129     | CONGESTED/LOOP
100701252       | 143        | 47         | 0.3287     | CONGESTED/LOOP
22010083        | 117        | 47         | 0.4017     | CONGESTED/LOOP
900199          | 128        | 58         | 0.4531     | CONGESTED/LOOP
900272          | 118        | 54         | 0.4576     | CONGESTED/LOOP
22010095        | 123        | 57         | 0.4634     | CONGESTED/LOOP
22010088        | 187        | 89         | 0.4759     | CONGESTED/LOOP
900241          | 140        | 69         | 0.4929     | CONGESTED/LOOP
22010086        | 120        | 60         | 0.5000     | CONGESTED/LOOP
175             | 178        | 93         | 0.5225     | CONGESTED/LOOP
```

Figure 27: Implementation and output: Camera diversity.

```
========= Time: 2026-02-06 17:25:30 =========
Cameras with HIGH Traffic (> 100) and LOW Diversity (< 0.6):
Camera          | Total      | Unique     | Ratio      | Status
-------------------------------------------------------------------------
100701266       | 212        | 51         | 0.2406     | CONGESTED/LOOP
22010086        | 406        | 98         | 0.2414     | CONGESTED/LOOP
22010085        | 383        | 115        | 0.3003     | CONGESTED/LOOP
100701252       | 237        | 90         | 0.3797     | CONGESTED/LOOP
100701267       | 234        | 89         | 0.3803     | CONGESTED/LOOP
100701265       | 137        | 54         | 0.3942     | CONGESTED/LOOP
100701253       | 213        | 84         | 0.3944     | CONGESTED/LOOP
22010043        | 111        | 44         | 0.3964     | CONGESTED/LOOP
100701269       | 185        | 79         | 0.4270     | CONGESTED/LOOP
100701263       | 131        | 58         | 0.4427     | CONGESTED/LOOP
```

```
========= Time: 2026-02-06 17:25:50 =========
Cameras with HIGH Traffic (> 100) and LOW Diversity (< 0.6):
Camera          | Total      | Unique     | Ratio      | Status
-------------------------------------------------------------------------
100701252       | 321        | 81         | 0.2523     | CONGESTED/LOOP
22010086        | 352        | 94         | 0.2670     | CONGESTED/LOOP
100701262       | 292        | 80         | 0.2740     | CONGESTED/LOOP
22010085        | 363        | 113        | 0.3113     | CONGESTED/LOOP
100701265       | 175        | 61         | 0.3486     | CONGESTED/LOOP
100701267       | 226        | 84         | 0.3717     | CONGESTED/LOOP
100701254       | 212        | 81         | 0.3821     | CONGESTED/LOOP
100701269       | 180        | 72         | 0.4000     | CONGESTED/LOOP
100701253       | 189        | 77         | 0.4074     | CONGESTED/LOOP
22010084        | 209        | 86         | 0.4115     | CONGESTED/LOOP
```

Figure 28: Diagram or recorded result: Camera diversity.

10 Practical 2: four-hash distinct counting

camera_passes.csv supplies CameraID and PlateNumber. Each plate is hashed with md5, sha1, sha224, and sha256; the maximum trailing-zero register for each hash is maintained. The report combines the four registers by their median and uses 2 raised to that median as its estimate. This reduces the influence of one exceptionally large register, but four hashes still have high variance. If the median of an even number of registers is their middle-pair average, its value can be half-integer and the estimate need not be an integer power of two.

Submitted sample	Exact distinct	Reported estimate	Absolute relative error
Global sample 1	1793	2896	61.5170%
Global sample 2	3318	4870	46.7752%

The exact reference is distinct().count(). Four registers provide a small sketch, while the exact reference requires work/state proportional to observed distinct items. The errors illustrate the weakness of this simple estimator; they are not a demonstrated accuracy guarantee or an inevitable fixed error range. The pipeline's exact-evaluation operations also consume resources, so the complete experiment cannot be described as constant-memory merely because the register summary is small. For expiring windows, a cumulative maximum register cannot simply forget old items. Recomputing a sketch for each window or using an explicitly window-aware structure is necessary. This distinction should be checked against the notebook before interpreting a cumulative result as a sliding-window estimate.

```
import sys
import os
import hashlib
import statistics
from pyspark import SparkContext, SparkConf
from pyspark.streaming import StreamingContext

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

conf = SparkConf().setAppName("FM_Median_Robust").setMaster("local[2]")

try:
    sc.stop()
except:
    pass

sc = SparkContext(conf=conf)
sc.setLogLevel("WARN")

ssc = StreamingContext(sc, 5)

input_directory = "file:///home/reza/HW4/stream_input"
dstream = ssc.textFileStream(input_directory)

HASH_FUNCS = ['md5', 'sha1', 'sha224', 'sha256']

def get_trailing_zeros(binary_str):
    if not binary_str: return 0
    s = str(binary_str)
    return len(s) - len(s.rstrip('0'))
```

Figure 29: Implementation and output: Practical 2: four-hash distinct counting.

```
def apply_fm_hashing(plate):
    zeros_list = []
    plate_encoded = plate.strip().encode('utf-8')
    for alg in HASH_FUNCS:
        h_obj = getattr(hashlib, alg)(plate_encoded)
        h_hex = h_obj.hexdigest()
        h_bin = bin(int(h_hex, 16))[2:]
        zeros_list.append(get_trailing_zeros(h_bin))
    return zeros_list

def parse_camera_pass(line):
    try:
        parts = line.split(',')
        if len(parts) < 3: return None
        camera_id = parts[1].strip()
        plate = parts[2].strip()
        return (camera_id, plate)
    except:
        return None

parsed_dstream = dstream.map(parse_camera_pass).filter(lambda x: x is not None)

WINDOW_DURATION = 30
SLIDE_DURATION = 10
windowed_dstream = parsed_dstream.window(WINDOW_DURATION, SLIDE_DURATION)

def process_fm(time, rdd):
    try:
        if not rdd.isEmpty():
            rdd.cache()

            actual_global = rdd.map(lambda x: x[1]).distinct().count()
            actual_per_cam = rdd.map(lambda x: (x[0], x[1])).distinct() \
                                .map(lambda x: (x[0], 1)).reduceByKey(lambda a,b: a+b).collectAsMap()
```

Figure 30: Implementation and output: Practical 2: four-hash distinct counting.

```
            fm_rdd = rdd.map(lambda x: (x[0], apply_fm_hashing(x[1])))

            def elem_max(l1, l2):
                return [max(a, b) for a, b in zip(l1, l2)]

            global_zeros = fm_rdd.map(lambda x: x[1]).reduce(elem_max)


            median_zeros_global = statistics.median(global_zeros)
            est_global = int(2 ** median_zeros_global)

            err_global = 0.0
            if actual_global > 0:
                err_global = abs(actual_global - est_global) / actual_global * 100

            print(f"========== Time: {time} ==========")
            print(f"Global Actual: {actual_global} | Estimated: {est_global} | Error: {err_global:.2f}%")
            print("-" * 60)


            cam_estimates = fm_rdd.reduceByKey(elem_max) \
                                  .mapValues(lambda z: int(2 ** statistics.median(z))) \
                                  .collect()

            cam_estimates.sort(key=lambda x: x[1], reverse=True)

            print("Camera Estimates (Top 10):")
            for cam, est in cam_estimates[:10]:
                act = actual_per_cam.get(cam, 0)
                err = 0.0
                if act > 0: err = abs(act - est) / act * 100
                print(f"Cam {cam:<5} | Act: {act:<5} | Est: {est:<5} | Err: {err:.2f}%")
            print("\n")

            rdd.unpersist()
    except Exception as e:
        print(e)
        pass

windowed_dstream.foreachRDD(process_fm)
```

Figure 31: Implementation and output: Practical 2: four-hash distinct counting.

```
========= Time: 2026-02-06 19:22:20 =========
Global Actual: 1793 | Estimated: 2048 | Error: 14.22%
------------------------------------------------------------

Camera Estimates (Top 10):
Cam 13     | Act: 83     | Est: 512    | Err: 516.87%
Cam 35     | Act: 39     | Est: 181    | Err: 364.10%
Cam 22     | Act: 64     | Est: 128    | Err: 100.00%
Cam 19     | Act: 65     | Est: 128    | Err: 96.92%
Cam 8      | Act: 34     | Est: 128    | Err: 276.47%
Cam 36     | Act: 32     | Est: 128    | Err: 300.00%
Cam 48     | Act: 72     | Est: 90     | Err: 25.00%
Cam 32     | Act: 60     | Est: 90     | Err: 50.00%
Cam 47     | Act: 52     | Est: 90     | Err: 73.08%
Cam 49     | Act: 30     | Est: 90     | Err: 200.00%


========= Time: 2026-02-06 19:22:30 =========
Global Actual: 3318 | Estimated: 5792 | Error: 74.56%
------------------------------------------------------------

Camera Estimates (Top 10):
Cam 13     | Act: 166    | Est: 512    | Err: 208.43%
Cam 32     | Act: 138    | Est: 362    | Err: 162.32%
Cam 3      | Act: 149    | Est: 362    | Err: 142.95%
Cam 19     | Act: 139    | Est: 362    | Err: 160.43%
Cam 22     | Act: 140    | Est: 256    | Err: 82.86%
Cam 35     | Act: 82     | Est: 256    | Err: 212.20%
Cam 41     | Act: 147    | Est: 256    | Err: 74.15%
Cam 39     | Act: 151    | Est: 256    | Err: 69.54%
Cam 48     | Act: 124    | Est: 181    | Err: 45.97%
Cam 46     | Act: 91     | Est: 181    | Err: 98.90%
```

Figure 32: Diagram or recorded result: Practical 2: four-hash distinct counting.

```
========= Time: 2026-02-06 19:22:40 =========
Global Actual: 4542 | Estimated: 5792 | Error: 27.52%
------------------------------------------------------------

Camera Estimates (Top 10):
Cam 39     | Act: 209   | Est: 1024  | Err: 389.95%
Cam 41     | Act: 226   | Est: 1024  | Err: 353.10%
Cam 35     | Act: 113   | Est: 724   | Err: 540.71%
Cam 48     | Act: 185   | Est: 724   | Err: 291.35%
Cam 3      | Act: 237   | Est: 512   | Err: 116.03%
Cam 13     | Act: 207   | Est: 512   | Err: 147.34%
Cam 19     | Act: 207   | Est: 362   | Err: 74.88%
Cam 32     | Act: 198   | Est: 362   | Err: 82.83%
Cam 28     | Act: 183   | Est: 362   | Err: 97.81%
Cam 22     | Act: 207   | Est: 256   | Err: 23.67%


========= Time: 2026-02-06 19:22:50 =========
Global Actual: 4543 | Estimated: 8192 | Error: 80.32%
------------------------------------------------------------

Camera Estimates (Top 10):
Cam 41     | Act: 241   | Est: 1024  | Err: 324.90%
Cam 39     | Act: 225   | Est: 724   | Err: 221.78%
Cam 48     | Act: 188   | Est: 724   | Err: 285.11%
Cam 3      | Act: 237   | Est: 512   | Err: 116.03%
Cam 28     | Act: 180   | Est: 512   | Err: 184.44%
Cam 35     | Act: 117   | Est: 362   | Err: 209.40%
Cam 19     | Act: 214   | Est: 362   | Err: 69.16%
Cam 22     | Act: 218   | Est: 181   | Err: 16.97%
Cam 32     | Act: 210   | Est: 181   | Err: 13.81%
Cam 46     | Act: 151   | Est: 181   | Err: 19.87%
```

Figure 33: Diagram or recorded result: Practical 2: four-hash distinct counting.

```
========== Time: 2026-02-06 19:23:40 ==========
Global Actual: 4533 | Estimated: 4096 | Error: 9.64%
------------------------------------------------------------

Camera Estimates (Top 10):
Cam 41     | Act: 251    | Est: 1024   | Err: 307.97%
Cam 13     | Act: 182    | Est: 512    | Err: 181.32%
Cam 6      | Act: 98     | Est: 512    | Err: 422.45%
Cam 3      | Act: 236    | Est: 362    | Err: 53.39%
Cam 39     | Act: 228    | Est: 362    | Err: 58.77%
Cam 28     | Act: 172    | Est: 362    | Err: 110.47%
Cam 5      | Act: 88     | Est: 362    | Err: 311.36%
Cam 22     | Act: 185    | Est: 256    | Err: 38.38%
Cam 15     | Act: 172    | Est: 256    | Err: 48.84%
Cam 21     | Act: 66     | Est: 256    | Err: 287.88%


========== Time: 2026-02-06 19:23:50 ==========
Global Actual: 3317 | Estimated: 4096 | Error: 23.49%
------------------------------------------------------------

Camera Estimates (Top 10):
Cam 41     | Act: 169    | Est: 1024   | Err: 505.92%
Cam 13     | Act: 128    | Est: 512    | Err: 300.00%
Cam 39     | Act: 154    | Est: 362    | Err: 135.06%
Cam 3      | Act: 175    | Est: 256    | Err: 46.29%
Cam 31     | Act: 69     | Est: 181    | Err: 162.32%
Cam 15     | Act: 119    | Est: 181    | Err: 52.10%
Cam 32     | Act: 132    | Est: 181    | Err: 37.12%
Cam 43     | Act: 46     | Est: 181    | Err: 293.48%
Cam 17     | Act: 46     | Est: 181    | Err: 293.48%
Cam 38     | Act: 154    | Est: 181    | Err: 17.53%
```

Figure 34: Diagram or recorded result: Practical 2: four-hash distinct counting.

Table 5: Distinct-count estimates and absolute relative errors in two recorded windows.

Window	**Actual distinct count**	**Four-register estimate**	**Absolute relative error (%)**
1	1730	2048	18.3815028902
2	3318	2896.309375740099	12.7091815630

11 Practical 3: directed camera-transition graph

Observations are grouped by plate and sorted by recorded time. Each consecutive pair with different cameras produces an observed directed transition; repeated observations at one camera are removed as self-transitions in the described design. The report states 50 nodes and 79310 edges. For a simple directed graph with 50 nodes and no loops, at most 2450 unique directed camera pairs exist. Consequently 79310 refers to transition events or parallel edges, not unique camera-pair edges. Preserve that distinction when computing degree, weights, or drawing a simple graph. Observed transitions do not reconstruct every intervening road segment.

```
import sys
import os
from pyspark import SparkContext, SparkConf

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

conf = SparkConf().setAppName("TrafficGraphConstruction").setMaster("local[2]")

try:
    sc.stop()
except:
    pass

sc = SparkContext(conf=conf)
sc.setLogLevel("WARN")

INPUT_FILE = "file:///home/reza/HW4/camera_passes.csv"

def parse_traffic_line(line):
    try:
        parts = line.split(',')
        if parts[0] == "RecordID" or parts[1] == "CameraID":
            return None

        camera_id = parts[1].strip()
        plate_number = parts[2].strip()
        record_time = parts[3].strip()

        return (plate_number, (record_time, camera_id))
    except:
        return None

raw_rdd = sc.textFile(INPUT_FILE)
parsed_rdd = raw_rdd.map(parse_traffic_line).filter(lambda x: x is not None)

grouped_rdd = parsed_rdd.groupByKey()
```

Figure 35: Implementation and output: Practical 3: directed camera-transition graph.

```python
def create_transitions(data):
    plate, sightings = data
    sorted_sightings = sorted(sightings, key=lambda x: x[0])

    links = []
    for i in range(len(sorted_sightings) - 1):
        src_cam = sorted_sightings[i][1]
        dst_cam = sorted_sightings[i+1][1]

        if src_cam != dst_cam:
            links.append((src_cam, dst_cam))

    return links

edges_rdd = grouped_rdd.flatMap(create_transitions)

adjacency_list = edges_rdd.groupByKey().mapValues(list)

total_nodes = edges_rdd.flatMap(lambda x: [x[0], x[1]]).distinct().count()
total_edges = edges_rdd.count()

print("\n" + "="*40)
print("Graph Construction Report")
print("="*40)
print(f"Total Nodes: {total_nodes}")
print(f"Total Edges: {total_edges}")
print("-" * 40)
print("Sample Adjacency List (First 5 nodes):")

for node, neighbors in adjacency_list.take(5):
    print(f"Camera {node} --> {neighbors[:10]} ... (Total: {len(neighbors)})")
print("="*40 + "\n")
```

Figure 36: Implementation and output: Practical 3: directed camera-transition graph.

```
========================================
Graph Construction Report
========================================
Total Nodes: 50
Total Edges: 79310
----------------------------------------
Sample Adjacency List (First 5 nodes):
[Stage 6:>                                                          (0 + 2) / 2]
Camera 14 --> ['4', '35', '4', '35', '38', '13', '31', '1', '18', '27'] ... (Total: 1946)
Camera 4 --> ['14', '35', '35', '47', '8', '19', '19', '41', '41', '49'] ... (Total: 1013)
Camera 12 --> ['3', '3', '28', '28', '15', '15', '15', '18', '48', '39'] ... (Total: 2790)
Camera 40 --> ['39', '36', '39', '39', '37', '32', '37', '18', '18', '50'] ... (Total: 1491)
Camera 1 --> ['26', '26', '47', '47', '18', '15', '47', '36', '2', '21'] ... (Total: 1867)
========================================
```

Figure 37: Diagram or recorded result: Practical 3: directed camera-transition graph.

12 PageRank with dangling nodes

Ranks start at 1/N and use beta=0.85 for ten iterations. All nodes are retained through rightOuterJoin. Each iteration computes dangling mass d=sum(ranks of zero-outdegree nodes) and adds beta*d/N to every node in addition to (1-beta)/N, under uniform redistribution. For weighted transitions, normalize by outgoing weight; for an unweighted graph, normalize by outdegree. The probability-preserving recurrence is r_new(v)=(1-beta)/N+beta*d/N+beta*sum_u r(u)*P(u -> v). Check nonnegativity, rank sum near one, and a residual/convergence criterion. Ten iterations alone do not prove convergence. The traffic-dataset rankings are recorded outputs. The stationary four-node calculation is a separate theoretical example.

```
import sys
import os
from operator import add
from pyspark import SparkContext, SparkConf

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

conf = SparkConf().setAppName("TrafficPageRank_Fixed").setMaster("local[2]")

try:
    sc.stop()
except:
    pass

sc = SparkContext(conf=conf)
sc.setLogLevel("WARN")

INPUT_FILE = "file:///home/reza/HW4/camera_passes.csv"

def parse_traffic_line(line):
    try:
        parts = line.split(',')
        if parts[0] == "RecordID" or parts[1] == "CameraID":
            return None
        camera_id = parts[1].strip()
        plate_number = parts[2].strip()
        record_time = parts[3].strip()
        if not camera_id:
            return None
        return (plate_number, (record_time, camera_id))
    except:
        return None

raw_rdd = sc.textFile(INPUT_FILE)
parsed_rdd = raw_rdd.map(parse_traffic_line).filter(lambda x: x is not None)

grouped_rdd = parsed_rdd.groupByKey()
```

Figure 38: Implementation and output: PageRank with dangling nodes.

```
def create_transitions(data):
    plate, sightings = data
    sorted_sightings = sorted(sightings, key=lambda x: x[0])
    links = []
    for i in range(len(sorted_sightings) - 1):
        src = sorted_sightings[i][1]
        dst = sorted_sightings[i+1][1]
        if src != dst:
            links.append((src, dst))
    return links

edges = grouped_rdd.flatMap(create_transitions)


links = edges.distinct().groupByKey().mapValues(list)
links.cache()


all_nodes = edges.flatMap(lambda x: [x[0], x[1]]).distinct()
total_nodes_count = all_nodes.count()

dangling_nodes = all_nodes.subtract(links.map(lambda x: x[0]))
dangling_count = dangling_nodes.count()
full_links = links.union(dangling_nodes.map(lambda x: (x, [])))
full_links.cache()

ranks = all_nodes.map(lambda x: (x, 1.0 / total_nodes_count))

ITERATIONS = 10
BETA = 0.85

print(f"\nTotal Nodes: {total_nodes_count}")
print(f"Dangling Nodes: {dangling_count}")
print("Starting PageRank Iterations...\n")

all_nodes_pair = all_nodes.map(lambda x: (x, None))
all_nodes_pair.cache()
```

Figure 39: Implementation and output: PageRank with dangling nodes.

```
for i in range(ITERATIONS):
    dangling_mass = full_links.join(ranks).filter(lambda x: len(x[1][0]) == 0).map(lambda x: x[1][1]).sum()
    base_probability = ((1 - BETA) + BETA * dangling_mass) / total_nodes_count
    contribs = full_links.join(ranks).flatMap(
        lambda x: [(dest, x[1][1] / len(x[1][0])) for dest in x[1][0]]
    )

    ranks = contribs.reduceByKey(add).rightOuterJoin(all_nodes_pair).mapValues(
        lambda x, base=base_probability: (x[0] if x[0] is not None else 0.0) * BETA + base
    )

top_cameras = ranks.takeOrdered(10, key=lambda x: -x[1])
```

Figure 40: Implementation and output: PageRank with dangling nodes.

```
Total Nodes: 50
Dangling Nodes: 0
Starting PageRank Iterations...

[Stage 44:=============================================================> (80 + 2) / 82]
========================================
Top 10 Cameras by PageRank (Beta=0.85)
========================================
1. Camera 48          : 0.020109
2. Camera 31          : 0.020109
3. Camera 36          : 0.020109
4. Camera 24          : 0.020109
5. Camera 4           : 0.020109
6. Camera 28          : 0.020109
7. Camera 29          : 0.020109
8. Camera 19          : 0.020109
9. Camera 40          : 0.020109
10. Camera 20          : 0.020109
========================================
```

Figure 41: Diagram or recorded result: PageRank with dangling nodes.

13 Rankings, count comparison, and visualization

The reported top ten are 48,31,36,24,4,28,29,19,40,20. The traffic-count top ten are 13,39,3,41,22,19,32,48,28,46. Their overlap is 48,28,19. Equal displayed ranks near 0.020109 can result from rounding, dense connectivity, or implementation choices; they do not prove structural symmetry or a main corridor.

Comparison	**Cameras**
Both lists	48,28,19
Rank only	31,36,24,4,29,40,20
Count only	13,39,3,41,22,32,46

A top-15 rank-selected subgraph is drawn with NetworkX/Matplotlib and line thickness based on transition counts. PageRank supplies a structural random-walk score, while observation totals supply volume. Neither score alone establishes a physical bottleneck, predicts disruption severity, or validates route planning. Such conclusions require topology, capacity, ground truth, and intervention evidence.

Table 7: Rankings, count comparison, and visualization - numerical data and parameters.

Ranking group	**Camera identifiers**	**Interpretation**
High volume and high recorded rank	19, 28, 48	Both recorded observation volume and structural score are high
High recorded rank only	31, 36, 24, 4, 29, 40, 20	The graph-based ranking differs from simple volume
High volume only	13, 39, 3, 41, 22, 32, 46	High observed volume does not by itself imply high graph rank

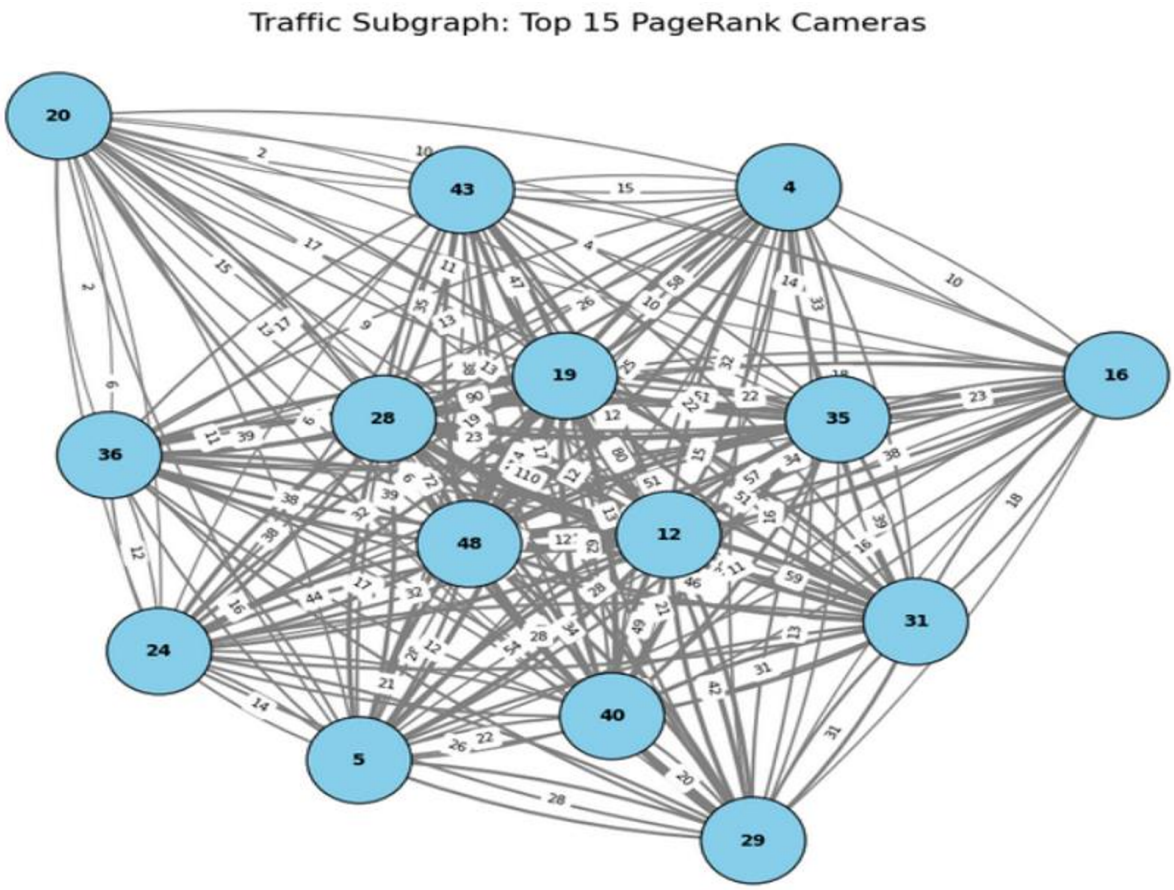

Figure 42: Diagram or recorded result: Rankings, count comparison, and visualization.

14 Four-node ranking results

Model, node order A/B/C/D	A	B	C	D
Uniform PageRank	0.324561404	0.225146199	0.225146199	0.225146199
Teleport A	0.403508772	0.198830409	0.198830409	0.198830409
Teleport A/C	0.373245614	0.183918129	0.258918129	0.183918129
Trust B	0.284903047	0.323453370	0.173453370	0.218190212
Raw spam mass	0.122190612	-0.436637047	0.229596719	0.030895420
HITS hubs, max normalized	1.000000000	0.391943596	0.102775049	0.710831454
HITS authority, max normalized	0.289168546	1.000000000	1.000000000	0.813606503

These checks apply to the theoretical four-node graph, not a rerun of the traffic graph.